

Despliegue del sitio web corporativo en servidor propio

Instalación y operación del proceso automatizado de publicación

DIRIGIDO A	Área de Tecnología de la Información — Fiberlux
PREPARADO POR	Twin Studios — proveedor de desarrollo del sitio web
VERSIÓN	1.0
FECHA	26 de agosto de 2026
SITIO	Sitio web corporativo de Fiberlux
RAMA DE PUBLICACIÓN	main
DESTINO	Raíz del docroot del servidor de Fiberlux
ALCANCE	Comprende únicamente lo que se ejecuta dentro del servidor de Fiberlux. La administración del repositorio y del gestor de contenido corresponde al proveedor y no requiere intervención del área de TI.

Contenido

1	Objeto
2	Modelo de despliegue
3	Consideraciones de seguridad
4	Requisitos del servidor
5	Coordinación con el proveedor
6	Procedimiento de instalación
7	Descripción del proceso automatizado
8	Monitoreo
9	Operación y escalamiento
10	Puntos críticos
11	Lista de verificación de puesta en marcha

1 Objeto

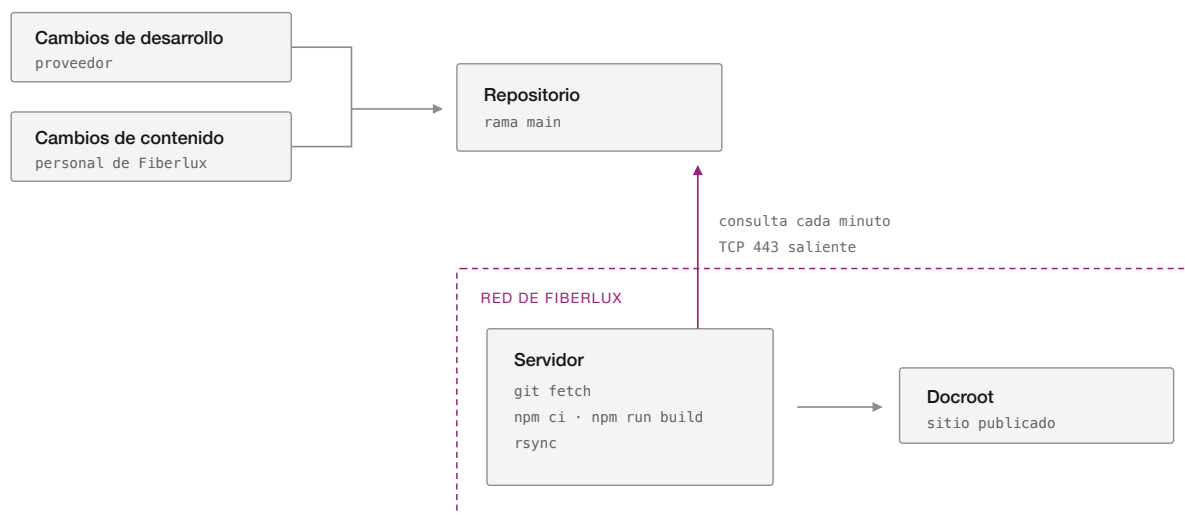
Este documento describe la instalación y operación del mecanismo por el cual el servidor de Fiberlux publica el sitio web corporativo de forma autónoma, sin recibir conexiones desde el exterior.

El esquema vigente delega la publicación en un servicio externo (GitHub Actions), que compila el sitio fuera de la red y accede al servidor por SFTP para depositarlo. Ese modelo obliga a exponer el servidor o a habilitar un túnel VPN, y a mantener credenciales de acceso al servidor almacenadas en un servicio de terceros.

El modelo que aquí se describe invierte la dirección de la comunicación: es el servidor de Fiberlux el que consulta el repositorio, compila el sitio con sus propios recursos y lo publica en su docroot. El proveedor no requiere acceso al servidor en ningún momento, ni durante la instalación ni en operación.

2 Modelo de despliegue

Un proceso programado consulta el repositorio cada minuto. Cuando detecta que la rama de publicación contiene cambios nuevos —originados en el trabajo de desarrollo del proveedor o en las ediciones de contenido que realiza el personal de Fiberlux desde el gestor— sincroniza la copia local, compila el sitio y actualiza el docroot.



Flujo de publicación. La consulta parte siempre del servidor de Fiberlux; ninguna conexión cruza el perímetro de la red hacia adentro, ni siquiera a través de una VPN.

La rama de publicación es `main` y el sitio se sirve desde la raíz del docroot. Ambos valores se configuran en los pasos 6.5 y 6.7.

3 Consideraciones de seguridad

ASPECTO	DESCRIPCIÓN
Procesos que se ejecutan	<code>git fetch</code> , <code>npm ci</code> , <code>npm run build</code> y <code>rsync</code> , invocados por una tarea programada cada minuto.
Usuario de ejecución	Usuario dedicado sin privilegios (<code>deploy</code>). No requiere <code>sudo</code> durante la operación.
Puertos entrantes	Ninguno. La totalidad del tráfico es saliente sobre TCP 443.
VPN o exposición del servidor	No se requieren. Su eliminación es el objeto de este cambio.
Credenciales de Fiberlux fuera de la red	Ninguna. Las credenciales SFTP actualmente almacenadas en el servicio externo se eliminan al concluir la migración.
Credenciales alojadas en el servidor	Una llave de solo lectura del repositorio y dos tokens de lectura del gestor de contenido. Todos revocables de forma inmediata.
Acceso externo al servidor	Ninguno. El servidor únicamente lee del repositorio.
Escritura en disco	Limitada a <code>/opt/fiberlux/</code> y al docroot del sitio.
Comportamiento ante fallo de compilación	No se publica nada. La sincronización con el docroot es la última operación y el proceso aborta antes de alcanzarla: el sitio en línea permanece intacto y el error queda registrado en el log.
Detención del proceso	Retirando la tarea programada del usuario <code>deploy</code> . Efecto inmediato, sin coordinación con terceros.
Ejecución de código de terceros	El proceso instala las dependencias del proyecto, como toda compilación de software de este tipo. El riesgo se acota mediante <code>npm ci</code> , que instala exactamente el listado de versiones fijado en el repositorio sin resolver versiones nuevas, y mediante la ejecución con un usuario sin privilegios.

4 Requisitos del servidor

4.1 Recursos

REQUISITO	DETALLE
Sistema operativo	Linux — Ubuntu 22.04 LTS, Rocky Linux 9 o equivalente.
Node.js	Versión 20 LTS. La compilación no está verificada en otras versiones.
Paquetes	<code>git</code> , <code>rsync</code> y <code>cron</code> (o temporizadores de <code>systemd</code>).
Almacenamiento	8 GB libres como mínimo. Distribución medida: dependencias 3 GB, repositorio con historial 0,7 GB, sitio compilado 140 MB, más el caché de paquetes.
Memoria	4 GB recomendados. La compilación constituye el pico de consumo. Con 2 GB se requiere espacio de intercambio y fijar <code>NODE_OPTIONS=--max-old-space-size=3072</code> .
Docroot	Permiso de escritura sobre el directorio que sirve el sitio.

4.2 Salidas de red requeridas

Las tres sobre TCP 443. No se requiere ninguna regla de entrada.

DESTINO	FINALIDAD
<code>github.com</code> , <code>codeload.github.com</code>	Clonado del repositorio y consulta de la rama.
<code>registry.npmjs.org</code>	Instalación de las dependencias del proyecto.
<code>*.tinajs.io</code> , <code>*.tina.io</code>	Descarga del contenido publicado desde el gestor de contenido.

4.3 Restricciones de egreso

Puerto 22 saliente bloqueado. Comunicarlo al proveedor. En lugar de una llave SSH se emite un token de solo lectura y el repositorio se clona por HTTPS. GitHub admite además SSH sobre el puerto 443 a través de `ssh.github.com`.

Sin acceso directo a `registry.npmjs.org`. Es admisible un repositorio interno intermedio (Verdaccio, Nexus o Artifactory) declarado en `.npmrc`. La compilación no puede, en cambio, prescindir de un origen de paquetes.

5 Coordinación con el proveedor

La puesta en marcha requiere tres intercambios entre ambas partes. Concluidos estos, el ciclo de publicación es autónomo y no exige coordinación adicional.

N.º	MOMENTO	ÁREA DE TI	PROVEEDOR
1	Paso 6.3	Remite la clave pública SSH generada en el servidor.	La registra como llave de solo lectura del repositorio y confirma su alta.
2	Previo al paso 6.5	Recibe las variables de compilación.	Las entrega por canal seguro: gestor de contraseñas corporativo o enlace de un solo uso.
3	Paso 6.8	Comunica que la primera publicación se completó correctamente.	Retira el despliegue por SFTP y elimina del servicio externo las credenciales del servidor.

Las variables del segundo intercambio son cinco: `TINA_CLIENT_ID`, `TINA_TOKEN`, `TINA_BRANCH`, `PUBLIC_TURNSTILE_SITE_KEY` y `DEPLOY_BASE`.

6 Procedimiento de instalación

Los pasos se ejecutan una sola vez y en el orden indicado.

6.1 Usuario y estructura de directorios

```
sudo useradd -m -s /bin/bash deploy
sudo mkdir -p /opt/fiberlux
sudo chown deploy:deploy /opt/fiberlux
sudo -iu deploy
```

6.2 Instalación de Node 20

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs git rsync
node -v      # debe imprimir v20.x
```

6.3 Generación de la llave de acceso al repositorio

```
ssh-keygen -t ed25519 -C "deploy-fiberlux-server" -f ~/.ssh/id_ed25519 -N ""
cat ~/.ssh/id_ed25519.pub
```

Se remite al proveedor exclusivamente la salida del comando `cat`, que corresponde a la clave pública. La clave privada permanece en el servidor. Debe esperarse la confirmación de alta antes de proceder al clonado.

La llave registrada es de solo lectura y habilita el acceso a ese repositorio y a ningún otro. El área de TI no requiere cuenta en la plataforma de alojamiento del código.

6.4 Clonado del repositorio

```
git clone -b main git@github.com:sagarkishnani/fiberlux-corp.git /opt/fiberlux/app
```

6.5 Variables de compilación

Se almacenan fuera del repositorio, con lectura restringida al usuario `deploy`.

```
cat > /opt/fiberlux/deploy.env <<'EOF'
TINA_CLIENT_ID=<entregado por el proveedor>
TINA_TOKEN=<entregado por el proveedor>
TINA_BRANCH=main
PUBLIC_TURNSTILE_SITE_KEY=<entregado por el proveedor>
DEPLOY_BASE=
EOF
chmod 600 /opt/fiberlux/deploy.env
```

`DEPLOY_BASE` se deja vacío: el sitio se publica en la raíz del docroot. Solo toma valor si el sitio colgara de un subdirectorio, supuesto en el cual el proveedor indicará el valor correspondiente.

Estas variables intervienen únicamente en la compilación. Las credenciales del servicio de correo se tratan en el paso siguiente.

6.6 Credenciales del servicio de correo

Los formularios del sitio —contacto, reclamos y libro de reclamaciones— son procesados por un componente PHP que requiere credenciales SMTP y las del panel de consulta de registros.

Residen en el archivo `fiberlux-config.php`, ubicado dentro del docroot y nunca en el repositorio.

Si el docroot es de nueva creación y aún no lo contiene, se genera a partir de la plantilla `public/config.example.php` incluida en el repositorio, completada con las credenciales de Fiberlux. El proceso de publicación está configurado para no sobrescribir este archivo en ninguna circunstancia.

6.7 Instalación del script de publicación

El script se distribuye versionado dentro del repositorio. Se copia fuera del clon, dado que cada publicación reescribe el árbol de trabajo.

```
cp /opt/fiberlux/app/scripts/server-deploy.sh /opt/fiberlux/deploy.sh
chmod +x /opt/fiberlux/deploy.sh
sudo touch /var/log/fiberlux-deploy.log
sudo chown deploy:deploy /var/log/fiberlux-deploy.log
```

Deben ajustarse las tres rutas de la cabecera de `/opt/fiberlux/deploy.sh`:

```
BRANCH="main"                # el valor por defecto del archivo es "staging"
REPO_DIR="/opt/fiberlux/app"
DOCR00T="<ruta real del docroot del sitio>"
```

VERIFICACIÓN OBLIGATORIA ANTES DE CONTINUAR

El parámetro **DOCR00T** es el más delicado de la instalación. La publicación se realiza mediante `rsync --delete`: todo archivo presente en ese directorio que no forme parte del sitio compilado será eliminado, con tres únicas excepciones (`data/`, `uploads/` y `fiberlux-config.php`).

El valor debe apuntar a un directorio que contenga exclusivamente este sitio. Si en él reside todavía otro contenido —por ejemplo una instalación de WordPress— la primera ejecución lo eliminaría.

6.8 Primera ejecución manual

Se realiza antes de programar la tarea. El parámetro `FORCE=1` es imprescindible: recién clonado, el repositorio local ya se encuentra en el último cambio, de modo que sin forzar la ejecución el script concluiría sin publicar y el docroot quedaría vacío.

```
sudo -iu deploy env FORCE=1 /opt/fiberlux/deploy.sh
tail -30 /var/log/fiberlux-deploy.log
```

Si la ejecución concluye con `publicado <sha>` y el sitio se visualiza correctamente, debe comunicarse al proveedor. Dicha comunicación habilita el retiro del despliegue por SFTP.

6.9 Programación de la tarea

```
sudo -iu deploy crontab -e
```

```
* * * * * /usr/bin/flock -n /tmp/fiberlux-deploy.lock /opt/fiberlux/deploy.sh
```

El uso de `flock -n` es obligatorio. Sin él, dos compilaciones simultáneas escribirían el docroot a la vez. Con él, las ejecuciones que se solapan con una publicación en curso se descartan.

6.10 Rotación del registro

```
sudo tee /etc/logrotate.d/fiberlux-deploy <<'EOF'
/var/log/fiberlux-deploy.log {
    weekly
    rotate 8
    compress
    missingok
    notifempty
    copytruncate
}
EOF
```

6.11 Alertas

Conforme a lo indicado en el apartado 8.

7 Descripción del proceso automatizado

7.1 Secuencia de ejecución

El script `/opt/fiberlux/deploy.sh` se ejecuta bajo `set -euo pipefail`: el fallo de cualquier etapa interrumpe las siguientes.

1. **Comparación.** Ejecuta `git fetch` y contrasta la copia local con la rama remota. Si coinciden y no se ha forzado la ejecución, concluye sin operar. Es el resultado de la práctica totalidad de las ejecuciones, cuyo coste es de milisegundos.
2. **Sincronización.** Aplica `git reset --hard` sobre la rama. No se emplea `pull`: la compilación escribe sobre archivos versionados y la operación fallaría por árbol de trabajo modificado.
3. **Carga de variables** desde `/opt/fiberlux/deploy.env`.
4. **Instalación de dependencias.** Ejecuta `npm ci` únicamente si varió el listado de dependencias o no existe el directorio `node_modules`. De ello depende que el ciclo habitual se resuelva en uno o dos minutos.
5. **Compilación.** `npm run build` genera el sitio estático completo.
6. **Publicación.** `rsync -a --delete` del sitio compilado al docroot.

7.2 Rutas preservadas

RUTA	MOTIVO DE LA PRESERVACIÓN
<code>data/</code>	Registros de los formularios enviados y contador de tickets.
<code>uploads/</code>	Archivos adjuntos remitidos por los usuarios del sitio.
<code>fiberlux-config.php</code>	Credenciales SMTP y del panel de consulta de registros.

Los archivos `.htaccess` de `data/` y `uploads/` sí se sincronizan: forman parte del sitio compilado y son los que impiden el acceso web a dichos directorios.

Dado que la sincronización con el docroot es la última operación de la secuencia, una compilación fallida no llega a publicarse en ningún caso: el sitio anterior continúa siendo servido y el error queda registrado.

7.3 Variante: servidor de compilación distinto del servidor web

Es una configuración admisible y preferible cuando el servidor web es una máquina reforzada en la que no se desea instalar Node ni habilitar salida a internet. Se sustituye la etapa de publicación del script por una sincronización sobre SSH dentro de la red, conservando las mismas exclusiones:

```
rsync -a --delete -e ssh \
  --include='/data/.htaccess' --exclude='/data/**' \
  --include='/uploads/.htaccess' --exclude='/uploads/**' \
  --exclude='/fiberlux-config.php' \
  ./dist/ deploy@servidor-web:<ruta del docroot>/
```

Requiere una clave SSH del usuario `deploy` autorizada en el servidor web. Este último recibe únicamente archivos estáticos: no precisa Node, ni `git`, ni salida a internet.

8 Monitoreo

El modo de fallo de este esquema no es la caída del servicio, sino la interrupción silenciosa de las publicaciones. El sitio permanecería en línea con la última versión publicada y el personal continuaría editando contenido sin advertir que no se difunde. Dos verificaciones sobre el registro son suficientes.

8.1 Continuidad de la tarea programada

El script escribe una línea en cada ejecución, haya cambios o no. Si el registro no se ha modificado en diez minutos, el proceso se ha detenido.

```
# alerta si el registro lleva más de 10 minutos sin escribirse
[ -z "$(find /var/log/fiberlux-deploy.log -mmin -10)" ] && echo "ALERTA: despliegue
Fiberlux detenido"
```

8.2 Fallos de compilación

Una publicación correcta se abre con `nuevo commit <sha>` y se cierra con `publicado <sha>`. La presencia de la primera línea sin la segunda indica una compilación fallida.

```
tail -100 /var/log/fiberlux-deploy.log | grep -c 'nuevo commit' # comparar contra...
tail -100 /var/log/fiberlux-deploy.log | grep -c 'publicado'    # ...este valor
```

Ambas verificaciones son integrables en la plataforma de supervisión que Fiberlux tenga en uso.

9 Operación y escalamiento

SITUACIÓN	RESPONSABLE	ACCIÓN
El registro muestra <code>publicado <sha></code> y el sitio se visualiza correctamente	—	Operación normal.
El registro no se actualiza	ÁREA DE TI	Verificar la tarea programada, el usuario <code>deploy</code> y el espacio disponible en disco.
Fallo de <code>git fetch</code>	ÁREA DE TI	Verificar la salida por TCP 443 y la llave SSH. Si la llave ha sido revocada, escalar al proveedor.
Fallo de <code>npm ci</code>	ÁREA DE TI	Verificar la salida hacia <code>registry.npmjs.org</code> o hacia el repositorio interno intermedio.
Fallo de <code>npm run build</code>	PROVEEDOR	Remitir las últimas cuarenta líneas del registro. La incidencia reside en el código y no en el servidor; el sitio en línea no resulta afectado.
El sitio no refleja un cambio de contenido	PROVEEDOR	Comprobar previamente que el registro no presente errores.
Necesidad de revertir a una versión anterior	PROVEEDOR	La reversión se practica sobre el repositorio y el servidor la aplica por sí mismo. No debe intervenir el servidor para revertir.
Mantenimiento o migración del servidor	ÁREA DE TI	Al restablecerse, la primera ejecución republica el estado vigente. No requiere aviso previo.
Rotación de la llave o de los tokens	AMBAS PARTES	El área de TI genera la nueva llave; el proveedor la registra y revoca la anterior.

10 Puntos críticos

Configuración de `DOCR00T`. Conforme a la advertencia del paso 6.7, un valor incorrecto elimina el contenido preexistente del directorio. Constituye el único riesgo de pérdida de datos del esquema y se circunscribe a la primera ejecución.

Edición del script dentro del clon. La tarea programada debe invocar `/opt/fiberlux/deploy.sh`, esto es, la copia situada fuera del árbol de trabajo. La operación `git reset --hard` reescribe el archivo del clon durante la ejecución, y el intérprete de órdenes, que lee el script por fragmentos, se interrumpe a mitad de proceso.

Duración de la primera instalación de dependencias. La descarga inicial ronda los 3 GB y puede prolongarse varios minutos en una conexión corporativa. Las ejecuciones posteriores omiten esta etapa salvo que varíe el listado de dependencias.

Rama por defecto del script. La cabecera del archivo versionado establece `BRANCH="${DEPLOY_BRANCH:-staging}"`. Debe fijarse en `main` (paso 6.7) y coincidir con `TINA_BRANCH=main` del archivo de variables (paso 6.5). Si ambos valores difieren, el sitio deja de

recibir los cambios de contenido sin que se produzca ningún error visible.

11 Lista de verificación de puesta en marcha

- ☐ Servidor aprovisionado con Linux, Node 20, `git`, `rsync` y `cron`.
 - ☐ 8 GB de almacenamiento libre y 4 GB de memoria, o espacio de intercambio configurado.
 - ☐ Salida TCP 443 habilitada hacia el repositorio, el registro de paquetes y el gestor de contenido.
 - ☐ Usuario `deploy` creado y `/opt/fiberlux` bajo su propiedad.
 - ☐ Clave pública SSH remitida al proveedor y alta confirmada.
 - ☐ Repositorio clonado en `/opt/fiberlux/app` sobre la rama `main`.
 - ☐ Archivo `deploy.env` creado con las cinco variables y permisos `600`.
 - ☐ Archivo `fiberlux-config.php` presente en el docroot.
 - ☐ Script copiado a `/opt/fiberlux/deploy.sh`, con `BRANCH="main"` y valor de `DOCROOT` verificado.
 - ☐ Primera ejecución con `FORCE=1` completada y sitio visible.
 - ☐ Comunicación cursada al proveedor.
 - ☐ Tarea programada activa con `flock`.
 - ☐ Rotación de registro configurada.
 - ☐ Alertas integradas en la plataforma de supervisión.
-